



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Tema 3.0: Desarrollo en Capas y Distribución del Procesamiento en Spring Boot

Resumen

Esta lectura conceptual aborda los fundamentos de la **arquitectura en capas lógica (N-Layer)** y su distinción con la **distribución física (N-Tier)** en el contexto de aplicaciones Java empresariales con **Spring Boot**. El objetivo es guiar al estudiantado para comprender el principio de separación de incumbencias (*separation of concerns*), estructurar correctamente un proyecto en capas (controlador, negocio, datos y dominio), emplear objetos de transferencia de datos (DTOs) para validar entradas y entender los beneficios de modularidad, mantenimiento paralelo y escalabilidad que ofrece este enfoque.

Concepto Principal (El 'Qué')

1. ¿Por qué emplear capas en el desarrollo de software?

Una aplicación de software compleja está conformada por múltiples clases y objetos que colaboran entre sí. Si todo el código (acceso a base de datos, validaciones, lógica matemática e interfaz de usuario) se mezcla en un único bloque, el sistema se vuelve rígido, difícil de mantener y propenso a errores colaterales; esto se conoce coloquialmente como **código espagueti o monolito acoplado**.

La **separación de incumbencias** consiste en organizar las clases según su especialización operativa. Las capas agrupan componentes que realizan tareas similares o complementarias, permitiendo establecer una jerarquía de dependencias unidireccional (las capas superiores consumen a las inferiores, pero nunca al revés).

2. Arquitectura Lógica (N-Layer) versus Física (N-Tier)

- **Arquitectura Lógica (N-Layer):** Es la segmentación del código fuente a nivel del diseño de la aplicación (paquetes y namespaces en Java, ej. `.controller`, `.business`, `.data`). Su principal objetivo es reducir la complejidad del desarrollo y facilitar el mantenimiento y las pruebas automatizadas. Ocurre dentro de la misma base de código.
- **Arquitectura Física (N-Tier):** Es la distribución de la ejecución del software en distintos componentes de infraestructura física, nodos de procesamiento o servidores en red (tiers). Por ejemplo, una arquitectura clásica de tres niveles físicos (3-tier) distribuye la carga en:
 - **Nivel de Cliente (Front-end):** Ejecutado en el navegador web del usuario (HTML/CSS/JS).
 - **Nivel de Aplicación (Back-end):** Servidor de aplicaciones web que ejecuta la lógica (Spring Boot).
 - **Nivel de Datos:** Motor gestor de base de datos relacional (ej. PostgreSQL, MySQL).

3. Responsabilidades de las Capas en Spring Boot

En la programación de APIs empresariales con Spring Boot, se sigue de manera estándar una estructura lógica de tres capas más una común: * **Capa de Controladores / Presentación (@RestController):** Recibe las solicitudes HTTP entrantes, valida la estructura de los datos del cliente, mapea las entradas mediante **Objetos de Transferencia de Datos (DTOs)** y retorna las respuestas con los códigos de estado HTTP idóneos (200, 201, 400, etc.). * **Capa de Negocio / Servicios (@Service):** Implementa las reglas y flujos de negocio de la aplicación, controla las transacciones de datos y coordina las llamadas a la capa de datos. * **Capa de Datos / Acceso a Datos (@Repository):** Interactúa directamente con la base de datos o almacén físico de

información. * **Capa de Dominio / Objetos Comunes (domain)**: Define el modelo de negocio básico y las entidades persistentes (compartidas entre todas las capas).

4. La Pirámide de Pruebas Automatizadas

Al separar la lógica en componentes desacoplados, se facilita la implementación de pruebas automatizadas: * **Pruebas Unitarias**: Validan clases aisladas en memoria sin cargar el contexto del framework (rápidas de ejecutar). * **Pruebas de Integración**: Verifican que la interacción entre múltiples componentes (ej. servicios y repositorios) funcione correctamente cargando el contexto de Spring. * **Pruebas de API / Web**: Evalúan los endpoints HTTP del controlador simulando llamadas externas.

Analogía (La Intuición)

- **La Organización de una Empresa (N-Layer)**: Piense en una empresa constructora grande. Internamente, el personal se divide en departamentos especializados (capas): Ventas (Presentación/Cliente), Ingeniería (Negocios) y Compras/Bodega (Capa de Datos). El vendedor de la oficina de ventas no diseña los planos estructurales ni va a la bodega a buscar cemento. En su lugar, envía la solicitud al ingeniero, quien calcula los materiales y le pide al bodeguero que despache el material. Cada uno es especialista en su departamento y la información fluye de manera ordenada.
- **Los Edificios Físicos de la Empresa (N-Tier)**: Imagine ahora que la oficina de ventas está en un local comercial en el centro de la ciudad (Nivel Cliente), el departamento de ingeniería y administración está en un edificio de oficinas en un parque tecnológico (Nivel Aplicación) y la bodega central de almacenamiento de materiales pesados está en una zona industrial fuera de la ciudad (Nivel de Datos). La división física en tres edificios independientes (tiers) permite que si la bodega se inunda, la oficina de ventas pueda seguir atendiendo clientes y recibiendo pedidos. Esto brinda escalabilidad física y tolerancia a fallas.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se muestra la estructuración de la funcionalidad **"Registrar Película"** del caso de estudio **VideoRent** organizada según el estándar multicapa de Spring Boot.

1. Objeto de Transferencia de Datos y Validación (`PeliculaCreationDTO.java`)

Cree el DTO para validar las entradas del cliente en la ruta:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/dto/PeliculaCreationDTO.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica2.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.util.List;

public class PeliculaCreationDTO {

    @NotBlank(message = "El título es obligatorio.")
    @Size(min = 1, max = 50, message = "El título debe medir 1 a 50 letras.")
    private String titulo;

    private boolean subtitulada;
    private boolean estreno;

    @NotNull(message = "El género es requerido.")
    private Integer generoId;

    @NotNull(message = "Debe ingresar al menos un actor.")
    @Size(min = 1, message = "Se requiere registrar al menos un actor.")
    private List<Integer> actorIds;

    // Getters y Setters
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }

    public boolean isSubtitulada() { return subtitulada; }
    public void setSubtitulada(boolean sub) { this.subtitulada = sub; }

    public boolean isEstreno() { return estreno; }
    public void setEstreno(boolean est) { this.estreno = est; }

    public Integer getGeneroId() { return generoId; }
    public void setGeneroId(Integer genId) { this.generoId = genId; }
```

```

    public List<Integer> getActorIds() { return actorIds; }
    public void setActorIds(List<Integer> actIds) { this.actorIds = actIds; }
}

```

2. Entidad del Dominio (Película.java)

Cree la entidad compartida en la ruta:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/domain/Pelicula.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.domain;

import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "peliculas")
public class Pelicula {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String titulo;
    private boolean subtitulada;
    private boolean estreno;

    @ManyToOne
    @JoinColumn(name = "genero_id")
    private Genero genero;

    @ManyToMany
    @JoinTable(
        name = "pelicula_actor",
        joinColumns = @JoinColumn(name = "pelicula_id"),
        inverseJoinColumns = @JoinColumn(name = "actor_id")
    )
    private List<Actor> actores;

    // Constructores, Getters y Setters...
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitulo() { return titulo; }

```

```
    public void setTitulo(String titulo) { this.titulo = titulo; }  
}
```

3. Capa de Datos / Repositorio (**PeliculaRepository.java**)

Declare el repositorio JPA en la ruta:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/data/PeliculaRepository.java

```
package cr.ac.ucr.paraiso.ie.carnet.practica2.data;  
  
import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Pelicula;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
@Repository  
public interface PeliculaRepository  
    extends JpaRepository<Pelicula, Integer> {  
    // Hereda las operaciones CRUD y consultas automatizadas  
}
```

4. Capa de Negocio / Servicio (**PeliculaService.java**)

Implemente las reglas y transacciones en la ruta:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/business/PeliculaService.java

```
package cr.ac.ucr.paraiso.ie.carnet.practica2.business;  
  
import cr.ac.ucr.paraiso.ie.carnet.practica2.data.PeliculaRepository;  
import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Pelicula;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;  
import org.springframework.transaction.annotation.Transactional;  
  
@Service  
public class PeliculaService {  
  
    @Autowired  
    private PeliculaRepository repository;  
  
    @Transactional  
    public Pelicula guardarPelicula(Pelicula pelicula) {
```

```

        // Aquí se inyectan las reglas de negocio (ej. validaciones)
        return repository.save(pelicula);
    }
}

```

5. Capa de Controladores / REST API (`PeliculaRestController.java`)

Exponga el endpoint HTTP en la ruta:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/controller/PeliculaRestController.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.controller;

import cr.ac.ucr.paraiso.ie.carnet.practica2.business.PeliculaService;
import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Pelicula;
import cr.ac.ucr.paraiso.ie.carnet.practica2.dto.PeliculaCreationDTO;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.BindingResult;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import java.util.List;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api/peliculas")
public class PeliculaRestController {

    @Autowired
    private PeliculaService service;

    @PostMapping
    public ResponseEntity<?> registrar(
        @Validated @RequestBody PeliculaCreationDTO dto,
        BindingResult validacion
    ) {
        // 1. Validar inputs e interceptar errores de restricciones DTO
        if (validacion.hasErrors()) {
            List<String> errores = validacion.getFieldErrors().stream()
                .map(err -> err.getField() + ": " + err.getDefaultMessage())
                .collect(Collectors.toList());
            return ResponseEntity.badRequest().body(errores);
        }
    }
}

```

```
// 2. Mapear DTO a Entidad del Dominio
Película nueva = new Película();
nueva.setTitulo(dto.getTitulo());
nueva.setSubtitulada(dto.isSubtitulada());
nueva.setEstreno(dto.isEstreno());

// 3. Invocar la capa de negocio y guardar
Película guardada = service.guardarPelícula(nueva);
return ResponseEntity.status(HttpStatus.CREATED).body(guardada);
}
}
```

Trampas Comunes

Exponer Entidades del Dominio Directamente en la API: Retornar o recibir las entidades de base de datos directamente del cliente de la API expone la estructura interna de la base de datos y puede permitir la inyección de atributos indeseados (*Overposting*).

- *Solución:* Use siempre **DTOs (Data Transfer Objects)** para la entrada y salida de datos del API de forma aislada.

Colocar Lógica de Negocio en el Controlador o Repositorio: Escribir condicionales de validación de negocio complejas en el `@RestController` o dentro de consultas personalizadas en el `@Repository` rompe la cohesión del diseño arquitectónico.

- *Solución:* Mantenga los controladores y repositorios limpios y delegue toda decisión conceptual a la capa `@Service`.

Dependencia en Sentencias SQL Rígidas: Escribir consultas SQL nativas y procesarlas manualmente (acoplando el backend a un motor de base de datos específico) dificulta la portabilidad del sistema.

- *Solución:* Utilice métodos de repositorio de Spring Data JPA u object query languages (JPQL) que abstraigan el dialecto del motor relacional.

Pregunta de Reflexión

- Si decidimos migrar nuestra aplicación monolítica de Spring Boot en capas a una arquitectura de **Microservicios**, ¿cómo facilita la separación lógica realizada en capas (N-Layer) esta transición física y organizativa del sistema?